Comprehensive Data Structures in C Reference

This repository contains robust, clean, and well-documented C implementations of fundamental and advanced data structures. Each implementation includes core operations, memory management, algorithmic invariants, and an interactive demo driver (`main`).

Table of Contents

1. Overview & Summary Table
2. Linear Data Structures
    - Stack (Array Implementation)
    - Queue (Linear Array Implementation)
    - Doubly Linked List
3. Tree-Based Data Structures
    - Binary Search Tree (BST)
    - AVL Tree (Self-Balancing BST)
    - B-Tree (Multi-Way Search Tree)
4. Advanced Priority Queues & Disjoint Sets
    - Fibonacci Heap
    - Disjoint Set Union (DSU / Union-Find)
5. Graph Representations & Traversals
    - Graph (Adjacency Matrix)
    - Graph (Adjacency List)
6. Compilation & Execution Guide

Overview & Summary Table

Data Structure	Source File	Category	Insert	Delete	Search / Lookup	Space
Stack	stack.c	Linear	$O(1)$	$O(1)$	$O(n)$	$O(MAX)$
Queue	queue.c	Linear	$O(1)$	$O(1)$	$O(n)$	$O(MAX)$
Doubly Linked List	doublyLinkedList.c	Linear	$O(1)$ beg / $O(n)$ end	$O(1)$ beg / $O(n)$ end	$O(n)$	$O(n)$
Binary Search Tree	binarySearchTree.c	Tree	Avg $O(\log n)$, Worst $O(n)$	Avg $O(\log n)$, Worst $O(n)$	Avg $O(\log n)$, Worst $O(n)$	$O(n)$
AVL Tree	avlTree.c	Self-Balancing Tree	$O(\log n)$ (Guaranteed)	$O(\log n)$ (Guaranteed)	$O(\log n)$ (Guaranteed)	$O(n)$

Data Structure	Source File	Category	Insert	Delete	Search / Lookup	Space
B-Tree	btree.c	Multi-Way Search Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Fibonacci Heap	fibonacciHeap.c	Priority Queue	$O(1)$ Amortized	$O(\log n)$ Extr-Min	$O(1)$ Min / $O(1)$ Decr-Key	$O(n)$
Disjoint Set Union	disjointset.c	Set / Partitioning	$O(1)$ makeSet	N/A	$O(\alpha(n)) \approx O(1)$	$O(n)$
Graph (Adj. Matrix)	graph.c	Graph	Edge: $O(1)$	Edge: $O(1)$	Edge: $O(1)$, Traversal: $O(V^2)$	$O(V^2)$
Graph (Adj. List)	GraphAdjList.c	Graph	Edge: $O(1)$	Edge: $O(\deg(u))$	Edge: $O(\deg(u))$, Traversal: $O(V+E)$	$O(V+E)$

1. Linear Data Structures

1.1 Stack (Array Implementation) — stack.c

Theoretical Concept

A **Stack** is a linear collection of elements that adheres to the **LIFO (Last-In, First-Out)** principle: the element pushed last is the first one removed. Access is restricted to a single position: the **top**.

- **Applications**: Expression evaluation/parsing (infix to postfix), function call stacks, undo mechanisms in editors, and syntax bracket checking.
- **Key Pointers**:
  - `top` indicates the current head index. Initially `top = -1`.
  - **Overflow**: Triggered when `top == MAX - 1`.
  - **Underflow**: Triggered when `top == -1`.

Data Structure Definition

```c
#define MAX 100

int stack[MAX]; // Fixed-size buffer
int top = -1;   // Index of the top-most item
```

Functions List & Explanations

Function Signature	Description & Algorithm
int isEmpty()	**Check for emptiness**: Returns `1` (`true`) if `top == -1`, otherwise `0`. Used as an underflow guard.
int isFull()	**Check for capacity limit**: Returns `1` (`true`) if `top == MAX - 1`, otherwise `0`. Used as an overflow guard.
void push(int item)	**Push element**: Checks `isFull()`. If false, increments `top` and stores `item` into `stack[top]`. Displays error message on overflow.
int pop()	**Pop element**: Checks `isEmpty()`. If false, retrieves `stack[top]`, decrements `top`, and returns the retrieved item. Returns `-1` on underflow.
int peek()	**Inspect top**: Checks `isEmpty()`. Returns the element at `stack[top]` without modifying `top`. Returns `-1` if empty.
void display()	**Traverse stack**: Iterates downwards from `top` down to index `0`, printing all active stack elements.
int main()	**Test driver**: Demonstrates push operations, top peeking, popping, state display, and emptiness checks.

1.2 Queue (Linear Array Implementation) — queue.c

Theoretical Concept

A **Queue** is a linear data structure adhering to the **FIFO (First-In, First-Out)** principle: elements are inserted at the back (**rear**) and removed from the front (**front**).

- **Applications**: CPU process scheduling (e.g., Round Robin), asynchronous request buffering, printer spooling, and Breadth-First Search (BFS).
- **Key Pointers**:
  - `front`: Tracks the index of the first valid element.
  - `rear`: Tracks the index of the latest inserted element.
  - Both initialized to `-1`.
  - When the last remaining element is dequeued (`front == rear`), both are reset to `-1`.

Data Structure Definition

```c
#define MAX 100

int queue[MAX]; // Fixed-size storage array
int front = -1; // Pointer to dequeue location
int rear = -1;  // Pointer to enqueue location
```

Functions List & Explanations

Function Signature	Description & Algorithm
int isEmpty()	**Check for emptiness**: Returns `1` if `front == -1`, indicating no elements are queued.
int isFull()	**Check for saturation**: Returns `1` if `rear == MAX - 1`, indicating no further insertions can take place in this linear layout.
void enqueue(int value)	**Enqueue element**: Validates with `isFull()`. If queue was empty (`front == -1`), sets `front = 0`. Increments `rear` and writes `value` to `queue[rear]`.
int dequeue()	**Dequeue element**: Validates with `isEmpty()`. Retrieves `queue[front]`. If `front == rear` (single element remaining), resets both `front` and `rear` to `-1`; otherwise advances `front++`. Returns extracted value.
int peek()	**Inspect front element**: Validates with `isEmpty()`. Returns `queue[front]` without advancing the pointer.
void display()	**Traverse queue**: Iterates from `front` to `rear` inclusive, printing the current queued items in FIFO order.
int main()	**Test driver**: Executes enqueue operations, front peeking, sequential dequeue operations, and post-dequeue enqueues.

1.3 Doubly Linked List — doublyLinkedList.c

Theoretical Concept

A **Doubly Linked List (DLL)** is a sequence of dynamically allocated nodes. Unlike singly linked lists, each node carries **two pointers**: one to the succeeding node (`next`) and one to the preceding node (`prev`).

- **Advantages**:
  - Bidirectional traversal (forward and backward).
  - Deleting a node takes $O(1)$ time if a pointer to that node is already known (no predecessor search required).
  - Insertion immediately before a specified node is straightforward.
- **Drawbacks**: Requires additional pointer overhead per node ($2 \times \text{pointer size}$).

Data Structure Definition

```c
typedef struct Node {
    int data;           // Payload value
    struct Node *prev;  // Link to preceding node (NULL if head)
    struct Node *next;  // Link to succeeding node (NULL if tail)
} Node;
```

Functions List & Explanations

Function Signature	Description & Algorithm
Node *createNode(int data)	**Node allocation**: Dynamically allocates a `Node` via `malloc`, initializes `data = data`, and sets `prev = NULL` and `next = NULL`. Validates allocation.
void insertAtBeginning(Node **head, int data)	**Prepend node**: Allocates a new node. Links `newNode->next = *head`. If list is non-empty, sets `(*head)->prev = newNode`. Updates `*head = newNode`. Takes $O(1)$ time.
void insertAtEnd(Node **head, int data)	**Append node**: If list is empty, sets `*head = newNode`. Otherwise traverses to the final node (`temp->next == NULL`), setting `temp->next = newNode` and `newNode->prev = temp`.
void insertAtPosition(Node **head, int data, int position)	**Arbitrary insertion**: Inserts at a 1-based index. Traverses to index `position - 1`. Re-wires `next` and `prev` pointers between predecessor and successor nodes. Validates out-of-bounds inputs.
void deleteAtBeginning(Node **head)	**Delete head**: Checks for empty list. Saves `*head` in temporary pointer, advances `*head = (*head)->next`, sets new head's `prev = NULL` (if not NULL), and frees original head. Takes $O(1)$ time.
void deleteAtEnd(Node **head)	**Delete tail**: If only one node exists, frees it and sets `*head = NULL`. Otherwise traverses to the last node, unlinks it by setting `temp->prev->next = NULL`, and frees the tail.
void deleteAtPosition(Node **head, int position)	**Delete by index**: Traverses to the target node at 1-based position. Splices it out by linking `temp->prev->next = temp->next` and `temp->next->prev = temp->prev` (if successor exists), then calls `free(temp)`.
Node *searchNode(Node *head, int target)	**Linear search**: Scans linearly from `head` comparing `temp->data == target`. Returns pointer to node if found, or `NULL`.
int countNodes(Node *head)	**List length**: Traverses from head to end, counting total number of active nodes.
void displayForward(Node *head)	**Forward printing**: Traverses using `next` pointers from `head` through to the end, printing elements in format `NULL <- [val] <-> ... -> NULL`.
void displayBackward(Node *head)	**Reverse printing**: Navigates to the terminal node, then uses `prev` pointers to print elements back to `head`.
int main()	**Test driver**: Exercises insertions at head/tail/intermediate positions, bidirectional traversal, searching, node counting, and deletions.

2. Tree-Based Data Structures

2.1 Binary Search Tree (BST) — binarySearchTree.c

Theoretical Concept

A **Binary Search Tree** is a hierarchical node-based binary tree maintaining the **BST Invariant**:

1. All keys in the left subtree are strictly **less** than the node's key.
2. All keys in the right subtree are strictly **greater** than the node's key.
3. Left and right subtrees are themselves binary search trees.

- **Traversals**:
  - **In-Order** (`Left -> Root -> Right`): Traverses keys in strictly ascending sorted order.
  - **Pre-Order** (`Root -> Left -> Right`): Useful for tree cloning and serialization.
  - **Post-Order** (`Left -> Right -> Root`): Useful for bottom-up node deletion / freeing.
- **Deletion Cases**:
  - *Case 1 (Leaf)*: Free node, replace parent reference with `NULL`.
  - *Case 2 (One Child)*: Replace node with its child subtree.
  - *Case 3 (Two Children)*: Find the **In-Order Successor** (smallest key in right subtree), copy its key into current node, and recursively delete the successor from the right subtree.

Data Structure Definition

```c
typedef struct Node {
    int key;            // Node key
    struct Node *left;  // Pointer to left child subtree
    struct Node *right; // Pointer to right child subtree
} Node;
```

Functions List & Explanations

Function Signature	Description & Algorithm
Node *createNode(int key)	**Node allocation**: Allocates a new node via `malloc`, sets `key = key`, `left = NULL`, and `right = NULL`.
Node *insertNode(Node *root, int key)	**BST insertion**: Recursively traverses left if `key < root->key`, or right if `key > root->key`. When reaching a `NULL` leaf spot, attaches a new node. Duplicates are rejected.
Node *searchNode(Node *root, int target)	**Search**: Compares `target` with `root->key`. Recursively checks left or right subtree based on value. Returns matching node pointer or `NULL`.
Node *findMin(Node *root)	**Find minimum**: Follows `left` pointers continuously until `node->left == NULL`. Returns the leftmost node.
Node *findMax(Node *root)	**Find maximum**: Follows `right` pointers continuously until `node->right == NULL`. Returns the rightmost node.
Node *deleteNode(Node *root, int key)	**Node deletion**: Locates node with `key`. Handles the 3 deletion cases (0 children, 1 child, 2 children using In-Order Successor). Returns modified subtree root.
void inOrder(Node *root)	**In-order traversal**: Recursively prints `left`, current `root->key`, then `right`. Yields sorted sequence.

Function Signature	Description & Algorithm
void preOrder(Node *root)	**Pre-order traversal**: Recursively prints `root->key`, `left`, then `right`.
void postOrder(Node *root)	**Post-order traversal**: Recursively prints `left`, `right`, then current `root->key`.
int main()	**Test driver**: Builds a multi-node BST, verifies all three traversals, performs min/max queries, searches keys, and demonstrates all three deletion scenarios.

2.2 AVL Tree (Self-Balancing BST) — avlTree.c

Theoretical Concept

An **AVL Tree** (invented by Adelson-Velsky and Landis) is a self-balancing binary search tree. For every node, the height difference between left and right subtrees—known as the **Balance Factor (BF)**—must be at most 1:

$$\text{BF} = \text{Height}(\text{Left}) - \text{Height}(\text{Right}) \in \{-1, 0, +1\}$$

When an insertion or deletion causes $|\text{BF}| > 1$, the invariant is restored using **rotations** in $O(1)$ time:

1. **Left-Left (LL) Case** ($\text{BF} > 1$ and key inserted into left child's left subtree):
   - Fixed by a single **Right Rotation** on the unbalanced node.
2. **Right-Right (RR) Case** ($\text{BF} < -1$ and key inserted into right child's right subtree):
   - Fixed by a single **Left Rotation** on the unbalanced node.
3. **Left-Right (LR) Case** ($\text{BF} > 1$ and key inserted into left child's right subtree):
   - Fixed by a **Left Rotation** on the left child, followed by a **Right Rotation** on the unbalanced node.
4. **Right-Left (RL) Case** ($\text{BF} < -1$ and key inserted into right child's left subtree):
   - Fixed by a **Right Rotation** on the right child, followed by a **Left Rotation** on the unbalanced node.

Because height is strictly maintained at $O(\log n)$, search, insertion, and deletion are **guaranteed** $O(\log n)$ even with sorted input data.

Data Structure Definition

```c
typedef struct Node {
    int key;            // Node key
    int height;         // Height of subtree rooted at this node
    struct Node *left;  // Left child pointer
    struct Node *right; // Right child pointer
} Node;
```

Functions List & Explanations

Function Signature	Description & Algorithm
int getHeight(Node *node)	**Subtree height**: Returns `node->height`, or `0` if `node == NULL`.
int max(int a, int b)	**Utility helper**: Returns the greater of two integers `a` and `b`.

Function Signature	Description & Algorithm
Node *createNode(int key)	**Node allocation**: Allocates node with `key = key`, `height = 1` (leaf node default), and `left = right = NULL`.
int getBalanceFactor(Node *node)	**Compute balance factor**: Returns `getHeight(node->left) - getHeight(node->right)`. Returns `0` for `NULL`.
Node *rightRotate(Node *y)	**LL Rotation**: Rotates node `y` right around child `x`. Reassigns `x->right` to `y` and `y->left` to subtree `T2`. Updates heights of `y` then `x`. Returns new subtree root `x`.
Node *leftRotate(Node *x)	**RR Rotation**: Rotates node `x` left around child `y`. Reassigns `y->left` to `x` and `x->right` to subtree `T2`. Updates heights of `x` then `y`. Returns new subtree root `y`.
Node *insertNode(Node *node, int key)	**Self-balancing insertion**: Performs standard recursive BST insertion, updates node height, evaluates BF, and applies LL, RR, LR, or RL rotation if $
Node *findMin(Node *root)	**Find minimum**: Follows `left` pointers to retrieve the node with the smallest key.
Node *deleteNode(Node *root, int key)	**Self-balancing deletion**: Standard BST deletion (handles 0, 1, or 2 children via successor). On unwinding recursion, updates height, calculates balance factor, and triggers necessary rotations.
Node *searchNode(Node *root, int target)	**Search**: Traverses left or right based on key comparisons in $O(\log n)$ time.
void inOrder(Node *root)	**In-order traversal**: Recursively outputs elements in ascending sequence.
void preOrder(Node *root)	**Pre-order traversal**: Recursively outputs `key`, `left`, `right`. Reveals root and subtree structural hierarchy.
void displayDetails(Node *root)	**Inspect node balance**: Recursively displays key, height, and balance factor for each node in the tree.
int main()	**Test driver**: Inserts sorted keys (which would skew a basic BST into an $O(n)$ chain), verifies rebalancing rotations, inspects node heights/BFs, searches, and tests deletion rebalancing.

2.3 B-Tree (Multi-Way Search Tree) — btree.c

Theoretical Concept

A **B-Tree** is a self-balancing search tree designed to optimize read and write operations on block-storage media (disks, file systems, databases). In a B-Tree, nodes may have more than two children and contain multiple sorted keys.

This implementation uses **Order 4 (minimum degree $t = 2$)**:

- Maximum keys per node: $\text{MAX_KEYS} = 2t - 1 = 3$
- Minimum keys per non-root node: $\text{MIN_KEYS} = t - 1 = 1$

- Maximum children per node: $\text{MAX_CHILDREN} = 2t = 4$

Key Invariants

1. All leaf nodes reside at the **exact same depth**.
2. Every internal node with k keys has exactly $k + 1$ children.
3. Keys within every node are stored in sorted ascending order.
4. Child subtree i holds keys strictly bounded between $\text{keys}[i-1]$ and $\text{keys}[i]$.
5. **Proactive Splitting**: During insertion down the tree, full child nodes are split *before* descending into them, avoiding cascading splits.

Data Structure Definition

```c
#define MAX_KEYS 3
#define MIN_KEYS 1
#define MAX_CHILDREN (MAX_KEYS + 1)

typedef struct BTreeNode {
    int keys[MAX_KEYS];                       // Array of stored keys
    struct BTreeNode *children[MAX_CHILDREN]; // Pointers to child nodes
    int numKeys;                              // Current number of keys
    int isLeaf;                               // 1 if leaf node, 0 otherwise
} BTreeNode;
```

Functions List & Explanations

Function Signature	Description & Algorithm
BTreeNode *createNode(int isLeaf)	**Node allocation**: Allocates a `BTreeNode`, initializes `isLeaf`, sets `numKeys = 0`, and clears all child pointers to `NULL`.
BTreeNode *search(BTreeNode *root, int key)	**B-Tree search**: Searches linearly through keys in current node. If match found, returns current node. If not found and node is leaf, returns `NULL`. Otherwise recurses into child subtree `children[i]`.
void splitChild(BTreeNode *parent, int index, BTreeNode *child)	**Child splitting**: Splits a full child node (having 3 keys). Allocates a new node `newChild`. Moves the rightmost key (`keys[2]`) and corresponding children to `newChild`. Promotes the median key (`keys[1]`) up into `parent->keys[index]`. Retains leftmost key in `child`.
void insertNonFull(BTreeNode *node, int key)	**Insert into non-full node**: If node is a leaf, inserts `key` into its proper sorted position by shifting larger keys right. If internal node, finds child subtree; if child is full, invokes `splitChild`, decides which of the split halves to enter, and recurses.
void insert(BTreeNode **root, int key)	**B-Tree insertion**: If tree is empty, creates leaf root. If root has MAX_KEYS, creates a new root, sets old root as `children[0]`, calls `splitChild(newRoot, 0, oldRoot)` (increasing tree height by 1),

Function Signature	Description & Algorithm
	and calls `insertNonFull`. Otherwise calls `insertNonFull` directly on root.
void traverse(BTreeNode *root)	**In-order traversal**: Iterates through keys $0 \dots \text{numKeys} - 1$, recursively traversing child subtree `children[i]` before printing `keys[i]`, followed by the final child subtree `children[numKeys]`. Prints all keys sorted.
void searchKey(BTreeNode *root, int key)	**Search helper**: Calls `search()` and prints whether key was found in the B-Tree.
int main()	**Test driver**: Inserts multiple keys causing internal node and root splits, traverses tree, and tests search queries.

3. Advanced Priority Queues & Disjoint Sets

3.1 Fibonacci Heap — fibonacciHeap.c

Theoretical Concept

Invented by Michael Fredman and Robert Tarjan in 1987, the **Fibonacci Heap** is an advanced priority queue composed of a collection of min-heap-ordered trees arranged in circular doubly linked lists.

Why It Outperforms Standard Binary Heaps

Operation	Binary Heap	Fibonacci Heap (Amortized)
Insert	$O(\log n)$	$O(1)$
Find-Min	$O(1)$	$O(1)$
Union / Merge	$O(n)$	$O(1)$
Decrease-Key	$O(\log n)$	$O(1)$
Extract-Min	$O(\log n)$	$O(\log n)$

This constant-time amortized Decrease-Key allows **Dijkstra's Shortest Path** and **Prim's Minimum Spanning Tree** algorithms to run in optimal $O(E + V \log V)$ time instead of $O(E \log V)$.

Core Structural Invariants

1. **Lazy Operations**: Insertions and heap merges simply splice nodes into the circular root list in $O(1)$ time without structural cleanup. Consolidation is deferred until `extractMin()`.
2. **Circular Doubly Linked Lists**: Root list and child lists are circular doubly linked, enabling $O(1)$ node insertion, removal, and list concatenation.
3. **Marking Rule & Cascading Cuts**:
   - `mark = 0` when unparented or newly attached.
   - When a parent loses its **first** child, `mark = 1`.

- When a parent loses its **second** child, it is cut from its parent, moved to the root list, and the cut propagates upward recursively (**Cascading Cut**).
- This ensures that a tree of degree k has at least F_{k+2} nodes (Fibonacci sequence bound), guaranteeing max node degree is $O(\log n)$.

Data Structure Definition

```c
typedef struct FibNode {
    int key;                    // Priority key (min-heap ordered)
    int degree;                 // Number of children
    int mark;                   // 1 if node lost a child since being
parented
    struct FibNode *parent;     // Pointer to parent node
    struct FibNode *child;      // Pointer to an arbitrary child node
    struct FibNode *left;       // Left sibling in circular list
    struct FibNode *right;      // Right sibling in circular list
} FibNode;

typedef struct FibHeap {
    FibNode *min;               // Pointer to root containing minimum key
    int n;                      // Total number of nodes in heap
} FibHeap;
```

Functions List & Explanations

Function Signature	Description & Algorithm
FibHeap *makeHeap(void)	**Initialize heap**: Allocates and initializes an empty `FibHeap` with `min = NULL` and `n = 0`.
FibNode *makeNode(int key)	**Initialize node**: Allocates a `FibNode` with `key = key`, `degree = 0`, `mark = 0`, `parent = child = NULL`, and self-referencing circular pointers (`left = right = this`).
static void insertAfter(FibNode *a, FibNode *x)	**List splicing**: Inserts node `x` immediately to the right of node `a` in a circular doubly linked list in $O(1)$ time.
static void removeFromList(FibNode *x)	**List removal**: Splices node `x` out of its circular list by joining its left and right neighbors. Resets `x->left = x->right = x`.
static void addToRootList(FibHeap *H, FibNode *x)	**Add to root list**: Unsets parent pointer, resets `mark = 0`, and splices `x` into heap `H`'s root list. Updates `H->min` if `x->key < H->min->key`.
FibNode *fibInsert(FibHeap *H, int key)	**Insert key**: Creates a node with `key`, adds to root list in $O(1)$ time, increments `H->n`, and returns node pointer.
FibNode *fibMinimum(FibHeap *H)	**Inspect minimum**: Returns pointer `H->min` in $O(1)$ worst-case time.

Function Signature	Description & Algorithm
FibNode *fibExtractMin(FibHeap *H)	**Extract minimum**: Removes `H->min`. Promotes all children of `H->min` into the root list. Re-links root list, decrements `n`, calls `consolidate(H)` to pair trees of equal degree, and returns extracted node. $O(\log n)$ amortized time.
void consolidate(FibHeap *H)	**Tree consolidation**: Scans roots using an auxiliary degree array (`table[64]`). Whenever two trees share degree d, the tree with the larger root is linked as a child under the smaller root (`FibHeap-Link`), incrementing degree, until all root degrees are unique.
void cut(FibHeap *H, FibNode *x, FibNode *y)	**Cut operation**: Removes child `x` from child list of `y`, decrements `y->degree`, and adds `x` to heap root list.
void cascadingCut(FibHeap *H, FibNode *y)	**Cascading cut**: If `y` is parented: if `y->mark == 0`, sets `y->mark = 1`; if `y->mark == 1`, cuts `y` from its parent `z` and recurses upward with `cascadingCut(H, z)`.
int fibDecreaseKey(FibHeap *H, FibNode *x, int newKey)	**Decrease key**: Validates `newKey <= x->key`. Updates `x->key = newKey`. If heap property is violated (`x->key < x->parent->key`), cuts `x` from parent and triggers `cascadingCut` on parent. Updates `H->min`. Takes $O(1)$ amortized time.
FibHeap *fibUnion(FibHeap *H1, FibHeap *H2)	**Heap merge**: Concatenates circular root lists of `H1` and `H2` in $O(1)$ time, updates overall min pointer, sets total node count `n = H1->n + H2->n`, and frees superseded header.
int main()	**Test driver**: Performs insertions, minimum extraction, amortized $O(1)$ key reduction (`fibDecreaseKey`), and priority drainage in sorted order.

3.2 Disjoint Set Union (DSU / Union-Find) — disjointset.c

Theoretical Concept

Disjoint Set Union (DSU), also called **Union-Find**, manages a collection of elements partitioned into non-overlapping (disjoint) subsets.

Core Operations

1. `makeSet(n)`: Initializes n singleton sets.
2. `find(x)`: Determines the representative/root of the subset containing x.
3. `unionSets(a, b)`: Merges the subsets containing a and b.

Critical Optimizations

- **Naive Implementation**: Can degenerate into long chains of depth $O(n)$, causing operations to take $O(n)$ time.
- **Optimization 1: Path Compression (in `find`)**: During traversal to the root, every visited node's parent pointer is updated to point directly to the root, flattening the tree structure.

- **Optimization 2: Union by Rank**: The root of the tree with smaller depth/rank is attached under the root of the tree with larger rank, preventing height growth.

Time Complexity

With both optimizations, any sequence of m operations over n elements runs in $O(m \cdot \alpha(n))$ time, where α is the **Inverse Ackermann function**. For any practical value ($n \leq 10^{80}$ atoms in the observable universe), $\alpha(n) \leq 4$. Therefore, operations run in **effectively $O(1)$ constant time**.

- **Applications**: Kruskal's Minimum Spanning Tree algorithm, cycle detection in undirected graphs, and dynamic connectivity in networks.

Data Structure Definition

```
int *parent;   // parent[i] stores parent of node i (or i if root)
int *rankArr;  // rankArr[i] stores upper bound on tree height
int *sz;       // sz[i] tracks total elements in component rooted at i
int numSets;   // Total count of disjoint partitions
```

Functions List & Explanations

Function Signature	Description & Algorithm
void makeSet(int n)	**Initialization**: Allocates arrays for `n` elements. Initializes `parent[i] = i`, `rankArr[i] = 0`, `sz[i] = 1`, and `numSets = n`.
int find(int x)	**Find with path compression**: Recursively traverses to root: `parent[x] = find(parent[x])`. Flattens tree structure on unwinding and returns the root.
int unionByRank(int a, int b)	**Union by rank**: Finds roots `rootA = find(a)` and `rootB = find(b)`. If identical, elements are already unified (returns `0`, detecting a cycle). Otherwise attaches smaller rank root to larger rank root (incrementing rank if tied), sums component sizes, decrements `numSets`, and returns `1`.
int isConnected(int a, int b)	**Connectivity query**: Evaluates `find(a) == find(b)`. Returns `1` if they belong to the same partition, otherwise `0`.
int getComponentSize(int x)	**Component size**: Finds root of `x` and returns `sz[root]`.
void freeSet()	**Cleanup**: Releases dynamically allocated memory for `parent`, `rankArr`, and `sz`.
int main()	**Test driver**: Initializes 7 disjoint sets, unifies elements, performs connectivity checks, monitors component sizes, and tracks set counts.

4. Graph Representations & Traversals

4.1 Graph (Adjacency Matrix) — graph.c

Theoretical Concept

An **Adjacency Matrix** represents a graph with V vertices using a 2D array of dimensions $V \times V$:

- `adjMatrix[u][v] = 1` if an edge exists between vertex u and vertex v.
- `adjMatrix[u][v] = 0` if no edge exists.
- For an **undirected graph**, the matrix is **symmetric**: `adjMatrix[u][v] == adjMatrix[v][u]`.

Trade-Offs

- **Pros**:
  - Edge lookup between any two vertices takes $O(1)$ time.
  - Adding or removing edges is $O(1)$.
  - Well-suited for **dense graphs** ($E \approx V^2$).
- **Cons**:
  - Requires $O(V^2)$ space regardless of the number of edges.
  - Iterating over all neighbors of a vertex takes $O(V)$ time.
  - Traversals take $O(V^2)$ time.

Data Structure Definition

```c
#define MAX_VERTICES 20

int adjMatrix[MAX_VERTICES][MAX_VERTICES]; // 2D edge connection grid
int visited[MAX_VERTICES];                 // Traversal visitation flags
int numVertices = 0;                       // Active vertex count
```

Functions List & Explanations

Function Signature	Description & Algorithm
void initGraph(int vertices)	**Initialize matrix**: Validates against `MAX_VERTICES`. Sets `numVertices` and zeroes out the 2D array `adjMatrix`.
void addEdge(int u, int v)	**Add undirected edge**: Bounds-checks vertices `u` and `v`. Sets `adjMatrix[u][v] = 1` and `adjMatrix[v][u] = 1`.
void displayMatrix()	**Print matrix**: Displays formatted 2D table representing edge connections across all vertices.
void dfsRec(int vertex)	**DFS recursive visitor**: Marks current vertex visited, prints it, and iterates over column index $0 \dots V-1$. Recursively calls `dfsRec(neighbor)` for any unvisited adjacent vertex.
void DFS(int startVertex)	**Depth-First Search**: Resets `visited` array, validates `startVertex`, and launches `dfsRec`. Explores deeply along branches using system call stack (LIFO). Takes $O(V^2)$ time.

Function Signature	Description & Algorithm
void BFS(int startVertex)	**Breadth-First Search**: Resets `visited` array. Uses a local FIFO queue (`queue[MAX_VERTICES]`). Enqueues `startVertex` and marks it visited. Iteratively dequeues vertex, prints it, and enqueues all unvisited neighbors. Explores level-by-level in $O(V^2)$ time.
int main()	**Test driver**: Constructs a 5-vertex undirected graph, displays its adjacency matrix, and executes DFS and BFS traversals.

4.2 Graph (Adjacency List) — GraphAdjList.c

Theoretical Concept

An **Adjacency List** represents a graph as an array of linked lists of size V. Each entry `adjLists[u]` points to a linked list storing all vertices directly connected to vertex u.

Trade-Offs

- **Pros**:
  - Highly memory-efficient for **sparse graphs** ($E \ll V^2$). Space complexity is $O(V + E)$ compared to $O(V^2)$ for an adjacency matrix.
  - Iterating over adjacent neighbors takes $O(\deg(u))$ time.
  - Edge addition takes $O(1)$ time by prepending to the linked list.
  - Traversals run in optimal $O(V + E)$ time.
- **Cons**:
  - Verifying if edge (u, v) exists takes $O(\deg(u))$ time in the worst case.

Data Structure Definition

```c
// Node representing an adjacent vertex in a neighbor list
typedef struct Node {
    int vertex;         // Neighbor vertex index
    struct Node *next;  // Next adjacent neighbor pointer
} Node;

// Graph structure
typedef struct Graph {
    int numVertices;    // Total vertices count
    Node **adjLists;    // Array of linked list head pointers
} Graph;
```

Functions List & Explanations

Function Signature	Description & Algorithm
Node *createNode(int vertex)	**Adjacency node allocation**: Allocates a list node for `vertex` and sets `next = NULL`.
Graph *createGraph(int vertices)	**Graph allocation**: Allocates `Graph` struct and an array of `vertices` node pointers. Sets each `adjLists[i] = NULL`.
int isValidVertex(Graph *graph, int vertex)	**Bounds validator**: Returns `1` if `graph != NULL` and `0 <= vertex < graph->numVertices`.
void addEdge(Graph *graph, int src, int dest)	**Add undirected edge**: Validates vertices. Creates node for `dest`, prepending to `graph->adjLists[src]`. Creates node for `src`, prepending to `graph->adjLists[dest]`. Runs in $O(1)$ time.
void displayGraph(Graph *graph)	**Print graph**: Iterates over each vertex index `0 \dots V-1`, printing its chain of adjacent neighbors in format `Vertex v: [head] -> n1 -> n2 -> NULL`.
void dfsRec(Graph *graph, int vertex, int visited[])	**DFS recursive visitor**: Marks vertex visited, prints it, and traverses its linked list. Recursively visits unvisited neighbors.
void DFS(Graph *graph, int startVertex)	**Depth-First Search**: Allocates visited array with `calloc`. Invokes `dfsRec(graph, startVertex, visited)` in $O(V + E)$ time. Frees visited array.
void BFS(Graph *graph, int startVertex)	**Breadth-First Search**: Allocates dynamic visited array and queue array. Enqueues `startVertex` and traverses level-by-level through neighbor linked lists in $O(V + E)$ time. Cleans up memory.
void freeGraph(Graph *graph)	**Memory deallocation**: Iterates through all V linked lists freeing each node, then frees the `adjLists` pointer array and the `Graph` struct. Prevents memory leaks.
int main()	**Test driver**: Builds graph, displays adjacency lists, executes DFS and BFS, and safely frees all allocated memory.

Compilation & Execution Guide

All implementations are written in standard C99/C11 and depend only on the standard C library (`stdio.h`, `stdlib.h`, `limits.h`). They can be compiled using `gcc` or `clang`.

Compiling Any File

```
# General syntax
gcc -Wall -Wextra -O2 <filename.c> -o <output_binary>
```

Source files in the repository:

- stack.c
- queue.c

- doublyLinkedList.c
- binarySearchTree.c
- avlTree.c
- btree.c
- fibonacciHeap.c
- disjointset.c
- graph.c
- GraphAdjList.c

Examples:

```
gcc -Wall -Wextra -O2 stack.c -o stack
gcc -Wall -Wextra -O2 queue.c -o queue
gcc -Wall -Wextra -O2 doublyLinkedList.c -o dll
gcc -Wall -Wextra -O2 binarySearchTree.c -o bst
gcc -Wall -Wextra -O2 avlTree.c -o avl
gcc -Wall -Wextra -O2 btree.c -o btree
gcc -Wall -Wextra -O2 fibonacciHeap.c -o fibheap
gcc -Wall -Wextra -O2 disjointset.c -o dsu
gcc -Wall -Wextra -O2 graph.c -o graph_mat
gcc -Wall -Wextra -O2 GraphAdjList.c -o graph_list
```

Running the Programs

```
./stack
./queue
./dll
./bst
./avl
./btree
./fibheap
./dsu
./graph_mat
./graph_list
```